CMPEN 441 / SWENG 311: Operating System and System Programming

Lab 10: Simulating Race Condition (20 Points)

Objectives

By completing this lab, students should be able to:

- Familiarize basic bash commands
- Use system calls for programming

Introduction

Definition

A race condition occurs when multiple processes/threads share the same data, and their uncontrollable execution orders cause the issued outputs.

Implementation and Explanations

- As a prerequisite for lab 10, you are required to go over the Pthread-related system calls in Lab 6.
- Study the following program example. In the program, there are two threads in the main process. One thread increases the shared data by 1, and the other thread decreases the shared data by 1. Technically, the ultimate data should be the same as the original shared data. However, when the sequence of program execution changes, the ultimate data will not be the same as the original shared data. You need to determine which sequence or mechanism impacts the correct outputs.

```c
#include<stdio.h>
#include<unistd.h>
#include<pthread.h>

int shared = 5;

void *func1(){
     int x = shared;
     x = x + 1;
     printf("Inside thread1.\n");
     sleep(1);
     shared = x;
     printf("The shared: %d\n", shared);
     //pthread_exit(&x);
}

void *func2(){
     int y =  shared;
     y = y - 1;
     printf("Inside thread2.\n");
     sleep(1);
     shared = y;
     printf("The shared: %d. \n", shared);
     //pthread_exit(&y);
}

int main(){
    pthread_t a_thread, b_thread;
    //void *result;
    pthread_create(&a_thread, NULL, func1, NULL);
    pthread_create(&b_thread, NULL, func2, NULL);
    pthread_join(a_thread, NULL);
    pthread_join(b_thread, NULL);
    printf("Inside main process.\n");
    //printf("Thread return: %p\n", &result);
    printf("The shared: %d\n", shared);
}
```

Lab instruction

Task 1:

Based on the above example, modify the program and make its output the same as the Sample output.
(Hint: the options for the modification might not be unique. However, if the program you modify generates the same as the sample output, your program will meet the

requirements.)

- Sample output:

```
cat628@E8WENG8813:~$ gcc lab1001_op1.c
cat628@E8WENG8813:~$ ./a.out
Inside thread1.
Inside thread2.
The shared: 4.
The shared: 6
Inside main process.
The shared: 6
```

- Sample output:

```
cat628@E8WENG8813:~$ nano lab1001_op2.c
cat628@E8WENG8813:~$ gcc lab1001_op2.c
cat628@E8WENG8813:~$ ./a.out
Inside thread2.
Inside thread1.
The shared: 4.
The shared: 6
Inside main process.
The shared: 6
```

Task 2:

Based on the above example, modify the program and make its output the same as the Sample output.
(Hint: the options for the modification might not be unique. If the program you modify generates the same as the sample output, your program will meet the requirements.)

- Sample output:

```
cat628@E8WENG8813: ~
cat628@E8WENG8813:~$ gcc lab1002.c
cat628@E8WENG8813:~$ ./a.out
Inside thread1.
The shared: 6
Inside thread2.
The shared: 5.
Inside main process.
The shared: 5
cat628@E8WENG8813:~$
```

Task 3:

Write a program. In the program, there are two threads in the main process. Each thread function is to get a counter by using a loop. The counter follows the conditions:

- The counter will continue counting by 1E3(Not include 1E3)
- The counter will continue counting by 1E4(Not include 1E4)
- The counter will continue counting by 1E5(Not include 1E5)
- The counter will continue counting by 1E6(Not include 1E6)
- Place the outputs and your brief explanations as comments on your program.

Sample output:

```
cat628@E8WENG8813: ~
cat628@E8WENG8813:~$ gcc lab1003.c
cat628@E8WENG8813:~$ ./a.out
Inside thread B.
The counter: 1000
Inside thread A.
The counter: 2000
Inside main process.
The counter: 2000
cat628@E8WENG8813:~$
```

Sample output:

```
cat628@E8WENG8813: ~
cat628@E8WENG8813:~$ gcc lab1003.c
cat628@E8WENG8813:~$ ./a.out
Inside thread B.
Inside thread A.
The counter: 10000
The counter: 20000
Inside main process.
The counter: 20000
cat628@E8WENG8813:~$
```

Sample output:

```
cat628@E8WENG8813: ~
cat628@E8WENG8813:~$ gcc lab1003.c
cat628@E8WENG8813:~$ ./a.out
Inside thread A.
Inside thread B.
The counter: 100000
The counter: 108064
Inside main process.
The counter: 108064
cat628@E8WENG8813:~$ gcc lab1003.c
cat628@E8WENG8813:~$ ./a.out
Inside thread A.
Inside thread B.
The counter: 100000
The counter: 129588
Inside main process.
The counter: 129588
cat628@E8WENG8813:~$ gcc lab1003.c
cat628@E8WENG8813:~$ ./a.out
Inside thread A.
Inside thread B.
The counter: 100000
The counter: 131586
Inside main process.
The counter: 131586
cat628@E8WENG8813:~$ gcc lab1003.c
cat628@E8WENG8813:~$ ./a.out
Inside thread A.
Inside thread B.
The counter: 100000
The counter: 126388
Inside main process.
The counter: 126388
cat628@E8WENG8813:~$
```

Sample output:

```
cat628@E8WENG8813: ~
cat628@E8WENG8813:~$ gcc lab1003.c
cat628@E8WENG8813:~$ ./a.out
Inside thread A.
Inside thread B.
The counter: 1000000
The counter: 1233239
Inside main process.
The counter: 1233239
cat628@E8WENG8813:~$ gcc lab1003.c
cat628@E8WENG8813:~$ ./a.out
Inside thread B.
Inside thread A.
The counter: 625189
The counter: 1049880
Inside main process.
The counter: 1049880
cat628@E8WENG8813:~$ gcc lab1003.c
cat628@E8WENG8813:~$ ./a.out
Inside thread A.
Inside thread B.
The counter: 884293
The counter: 1576610
Inside main process.
The counter: 1576610
cat628@E8WENG8813:~$ gcc lab1003.c
cat628@E8WENG8813:~$ ./a.out
Inside thread B.
Inside thread A.
The counter: 1024422
The counter: 1146015
```

SUBMISSION

Submit four C source code files. According to the sequence of tasks, the submission follows:

- the first C source code file is called "**Lab1001_YourFirstnameLastname.c**".
- the second C source code file is called "**Lab1002_YourFirstnameLastname.c**".
- the third C source code file is called "**Lab1003_YourFirstnameLastname.c**.

Submitting files to the Dropbox of Canvas.